BÖLÜM 1 / MODULE 1

Temel İlkeler, Ölçeklenebilirlik ve Mülakat Çerçevesi

Core Fundamentals, Scalability & The MAANG Interview Framework

MAANG mülakatlarında başarılı olmanın 4 adımlı altın çerçevesi, dikey/yatay ölçekleme, P99 gecikme metrikleri, SLA/SLO ve CAP/PACELC teoremleri.

The 4-step MAANG interview blueprint, vertical vs horizontal scaling, tail latencies (P99), high availability math (SLA 9s), and CAP/PACELC trade-offs.

1

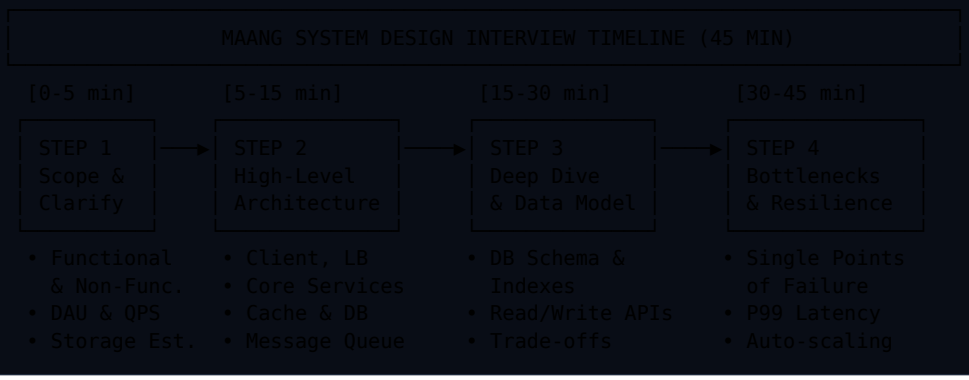
BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

4 Adımlı MAANG Mülakat Çerçevesi (RADAR Metodu)

The 4-Step MAANG Interview Blueprint (RADAR Method)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir inşaat mühendisi mülakatında 'Bana bir bina yap' dediklerinde hemen elinize tuğla alıp harç karmaya başlamazsınız! 'Kaç kişilik bir bina? 2 katlı bir yazlık mı, yoksa 80 katlı rüzgara dayanıklı bir gökdelen mi?' diye sorarsınız. MAANG mülakatı da böyledir: Mülakatçı size bilerek muğlak bir soru sorar ('Twitter tasarla'). Hemen kod yazmaya başlarsanız elenirsiniz! Önce gereksinimleri netleştirir (Scope), ardından kaba taslağı çizer (High-Level), kritik parçaların röntgenini çeker (Deep Dive) ve sistemi zorlayıp darboğazları çözersiniz (Bottlenecks)!

🇬🇧 ENGLISH (ELI5)

If a master architect is asked: 'Build me a structure', they don't immediately start mixing cement! They ask: 'Is this a 2-family wooden cabin or an 80-story earthquake-proof skyscraper?' MAANG interviews operate on the exact same premise: interviewers give intentionally vague prompts ('Design Twitter'). If you dive straight into drawing databases, you fail immediately. You must follow the 4 steps: clarify the scope, outline high-level blocks, deep-dive into bottlenecks, and harden failure domains!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

45 dakikalık MAANG sistem tasarımı mülakatının altın zaman planı: 1) Kapsam & Gereksinimler (5 dk): Fonksiyonel (Tweet atma, akış okuma) ve Fonksiyonel Olmayan (100M DAU, P99 < 200ms, 99.99% erişilebilirlik, read-heavy 100:1) hedefler. 2) Yüksek Seviye Mimari (10 dk): İstemci, DNS, CDN, LB, API Gateway, Mikroservisler ve Depolama blokları. 3) Derinlemesine Tasarım (15 dk): Veri şeması, NoSQL vs SQL seçimi, önbellek katmanı ve fan-out stratejisi. 4) İyileştirme & Darboğazlar (15 dk): SPOF (Single Point of Failure), partition stratejisi, rate limiter ve felaket kurtarma.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The rigorous 45-minute MAANG system design framework: 1) Scope & Clarification (5 min): Differentiate functional vs non-functional constraints (100M DAU, Read-to-Write ratio 100:1, P99 < 200ms, 99.99% availability). 2) High-Level Architecture (10 min): Map clients, DNS, CDN, L4/L7 Load Balancers, Gateways, Stateless Services, Cache, and Persistence. 3) Core Deep Dive (15 min): Schema design, data models, sharding keys, and query paths. 4) Bottlenecks & Operational Resiliency (15 min): Mitigate Single Points of Failure (SPOFs), backpressure, and cross-datacenter failover.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Mülakat Metriklerini ve SLA Değerlerini Temsil Eden Record Modeli
public record SystemRequirement(
    String systemName,
    long dailyActiveUsers,
    int readWriteRatio,
    double targetAvailabilityPercentage,
    int targetP99LatencyMillis
) {
    public long calculateDailyReadQps() {
        long dailyWrites = dailyActiveUsers * 2; // Kullanıcı başı 2 yazma
        long dailyReads = dailyWrites * readWriteRatio;
        return dailyReads / 86_400; // Saniyelik ortalama QPS
    }

    public long calculatePeakQps() {
        return calculateDailyReadQps() * 2; // Zirve yük çarpanı: 2x
    }
}

// Java 25 Main metodu ile çalıştırma:
void main() {
    var twitterReq = new SystemRequirement("Twitter Feed", 200_000_000L, 100, 99.99, 150);
    IO.println("Ortalama Okuma QPS: " + twitterReq.calculateDailyReadQps());
    IO.println("Zirve (Peak) Okuma QPS: " + twitterReq.calculatePeakQps());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

En ölümcül mülakat hatası: Soru sorulur sorulmaz sessiz kalıp doğrudan çizim yapmaya başlamaktır. Mülakat bir monolog değil, gelecekteki iş arkadaşınızla yaptığınız mimari bir istişaredir (collaborative dialogue).

🇬🇧 INTERVIEW TRAP & BOTTLENECK

The single most fatal mistake is jumping straight into drawing database boxes in absolute silence. A system design round is an interactive architectural collaboration, not an exam paper.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- İlk 5 dakikayı mutlaka fonksiyonel ve non-fonksiyonel gereksinimleri netleştirmeye ayırın.
- Tasarımı bir kerede mükemmel yapmaya çalışmayın; basitten başlayıp ölçekleyin.
- Her mimari kararın arkasında bir 'Trade-off' (ödünleşim) gerekçesi sunun.

🎯 Key Takeaways for the Interview

- Spend the initial 5 minutes strictly clarifying functional bounds and non-functional SLAs.
- Start with an intuitive baseline architecture, then evolve it under simulated scale.
- Always articulate the trade-offs and rationale behind every database or caching choice.

2

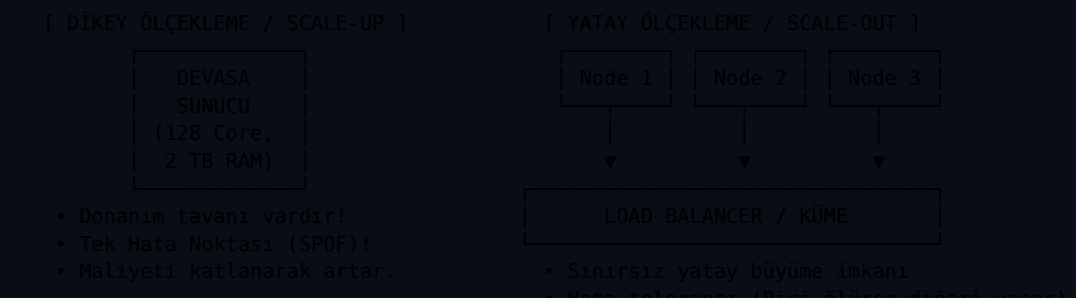
BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

Dikey vs Yatay Ölçekleme (Scale-Up vs Scale-Out) & Donanım Sınırları

Vertical vs Horizontal Scaling & Hardware Bottlenecks

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Diyelim ki bir pizzacınız var ve siparişlere yetişemiyorsunuz. Dikey ölçekleme (Scale-Up), dükkana dünyanın en kaslı, 8 kollu süper aşçısını transfer etmektir. Bir yere kadar harikadır ama o aşçı da yorulur, hasta olursa dükkan kapanır ve maaşı bir servettir! Yatay ölçekleme (Scale-Out) ise mutfağa standart 10 tane normal aşçı koyup siparişleri aralarında paylaştırmaktır. Aşçılardan biri bayılsa bile diğer 9 aşçı pizza yapmaya devam eder!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling pizzeria running out of oven capacity. Vertical scaling (Scale-Up) is hiring an expensive superhero chef with 8 arms. It works initially, but even superhuman chefs have physical limits, and if they catch the flu, your pizzeria goes bankrupt! Horizontal scaling (Scale-Out) is hiring 10 ordinary cooks and distributing orders across them. If cook 3 twists an ankle, the other 9 keep cooking seamlessly!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dikey ölçekleme (Scale-Up), sunucunun CPU, RAM veya SSD kapasitesini artırmaktır. Avantajı: Kod değişikliği gerektirmez, dağıtık sistem karmaşıklığı yoktur. Dezavantajı: Donanım sınırları (fizik kuralları), üstel maliyet artışı ve Tek Hata Noktası (SPOF). Yatay ölçekleme (Scale-Out), sisteme daha fazla ucuz/standart emtia makine (commodity hardware) eklemektir. Durumsuz (stateless) servisler için kusursuzdur; ancak veritabanlarında veri tutarlılığı, replikasyon ve ağ gecikmesi karmaşıklığı getirir.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Vertical scaling (Scale-Up) augments an existing node with more cores, RAM, and NVMe drives. Pros: Zero distributed networking overhead and seamless operational simplicity. Cons: Hard hardware ceilings, exponential cloud cost curves, and single point of failure (SPOF). Horizontal scaling (Scale-Out) pools clusters of commodity nodes behind load balancers. Trivial for stateless application tiers, but introduces distributed consensus, sharding, and network partition complexities on stateful databases.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Virtual Threads (Project Loom) ile Yatay İşçi Havuzu Simülasyonu
import java.util.concurrent.*;

public class HorizontalWorkerPool {
    public static void main(String[] args) throws InterruptedException {
        // Java 25: Milyonlarca hafif sanal iş parçacığı (Virtual Threads)
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 1; i <= 5; i++) {
                final int workerId = i;
                executor.submit(() -> {
                    IO.println("İşçi Düğüm #" + workerId + " görev yürütüyor: "
                        + Thread.currentThread());
                });
            }
        } // Executor otomatik kapanır ve tüm görevlerin bitmesini bekler
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Veritabanını hemen 100 parçaya bölerim (Sharding)' demeyin! Önce Read-Replica, Cache (Redis) ve Dikey Büyütme alternatiflerini tüketip, sharding'in getirdiği iki fazlı onay (2PC) ve cross-partition join zorluklarını bildiğinizi vurgulayın.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not scream 'Database Sharding!' at minute 10. Sharding introduces massive operational overhead like cross-partition joins and re-balancing. Always exhaust Read Replicas, Caching, and Vertical headroom first.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

- Uygulama katmanı (Stateless) daima yatay ölçeklenmelidir.
- Stateless application tiers should always be scaled horizontally behind load balancers.
- Veritabanı katmanında dikey ölçekleme sınırına ulaşılınca Okuma Replikaları ve ardından Sharding devreye girer.
- Scale databases via Read Replicas and caching before resorting to horizontal sharding.
- Yatay ölçekleme hata izolasyonu (fault isolation) sağlayarak sistemi çökmelere karşı korur.
- Horizontal scaling unlocks true fault tolerance by eliminating single points of failure.

3

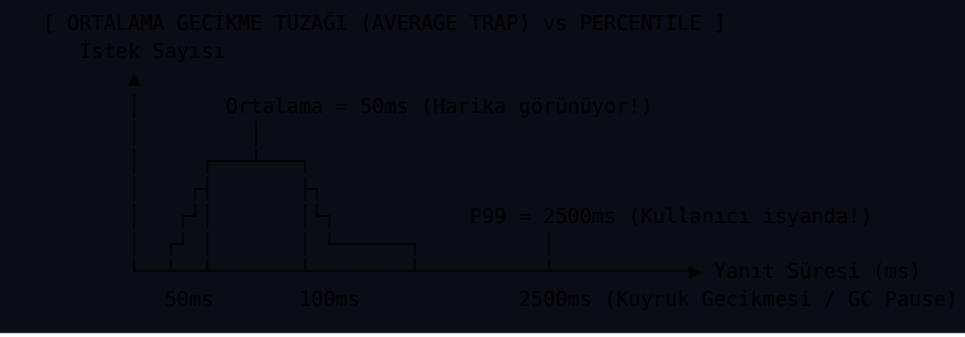
BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

Gecikme (Latency), Verim (Throughput) ve Ağ Temelleri (P99 Yanılgısı)

Latency, Throughput & Tail Latency (The P99 Fallacy)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir havaalanı pasaport kuyruğunu düşünün. 100 yolcudan 99'u pasaporttan 10 saniyede geçiyor; ancak 1 yolcu şüpheli valiz araması yüzünden 45 dakika bekliyor. Havaalanı müdürü 'Ortalama bekleme süremiz sadece 30 saniye!' derse bu istatistik gerçeği yansıtır mı? O 45 dakika bekleyen yolcu Twitter'da olay çıkarır! MAANG şirketleri ortalamaya (Average) asla inanmaz; P99'a (yani en şanssız %1'lik kullanıcının yaşadığı gecikmeye) bakar!

🇬🇧 ENGLISH (ELI5)

Consider an airport security checkpoint. 99 travelers breeze through in 10 seconds, but 1 unlucky passenger gets held for a 45-minute bag search. If the airport director claims: 'Our average wait time is only 35 seconds!', it masks a nightmare! That stalled passenger will rage on social media. MAANG engineers never track simple averages; they obsess over P99 and P99.9 (the tail latency suffered by the unluckiest 1% of users)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Gecikme (Latency), bir isteğin istemciden çıkıp sunucuda işlenip geri dönene kadar geçen süredir (RTT + processing). Verim (Throughput / QPS), sistemin birim zamanda tamamladığı işlem sayısıdır. Ortalama gecikme (Mean), uzun kuyruk gecikmelerini (Tail Latency), JVM GC duraklamalarını (Stop-the-World) ve disk IO darboğazlarını gizler. Modern bir web sayfasında 100 alt mikroservis çağrısı yapılıyorsa, ortalama bir kullanıcının en az 1 kez P99 gecikmesine çarpma olasılığı: 1 - (0.99)^100 = %63.4'tür! Yani kullanıcıların üçte ikisi yavaşlığa maruz kalır.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Latency is the elapsed duration for a round-trip request (network RTT + queueing + execution). Throughput (QPS/RPS) is system processing capacity per unit of time. The average (mean) latency is dangerously misleading because it masks tail latency spikes caused by garbage collection, lock contention, and TCP packet drops. If a web request aggregates calls across 100 microservices, the probability that a single end-user experiences a P99 tail latency is 1 - (0.99)^100 ≈ 63.4%! Tail latencies dominate user perception.

⚡ Modern Java 25 Uygulaması (Loom & Records) / ⚡ Java 25 Implementation

```java
// Java 25: Percentile (P50, P90, P99) Hesaplayıcı Motoru
import java.util.*;

public class LatencyPercentileTracker {
    private final List<Long> latencies = new ArrayList<>();

    public synchronized void recordLatency(long durationMillis) {
        latencies.add(durationMillis);
    }

    public synchronized long getPercentile(double percentile) {
        if (latencies.isEmpty()) return 0;
        List<Long> sorted = new ArrayList<>(latencies);
        Collections.sort(sorted);
        int index = (int) Math.ceil((percentile / 100.0) * sorted.size()) - 1;
        return sorted.get(Math.max(0, index));
    }
}

void main() {
    var tracker = new LatencyPercentileTracker();
    // 99 hızlı istek (10-30ms), 1 adet felaket yavaş istek (2500ms)
    for (int i = 0; i < 99; i++) tracker.recordLatency(20);
    tracker.recordLatency(2500);

    IO.println("P50 (Medyan): " + tracker.getPercentile(50) + " ms");
    IO.println("P99 (Tail):   " + tracker.getPercentile(99) + " ms");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı / ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçınız 'Sistem yanıt süreleriniz nasıl?' dediğinde 'Ortalama 50ms' deyip geçerseniz Senior seviyesinin altında kalırsınız. 'P50 medyanımız 30ms, P99 kuyruk gecikmemiz ise 120ms altında garanti edilir' şeklinde yanıt verin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Reporting simple average latencies during system design flags you as a junior engineer. Always discuss percentiles: 'Our median P50 is 25ms, while our P99 tail latency is bounded below 150ms.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar / 🎯 Key Takeaways for the Interview

- Ortalama gecikme (Mean), sistemdeki darboğazları ve memnuniyetsiz kullanıcıları gizler.
- Yüzlerce mikroservisin olduğu mimarilerde P99 gecikmesi tüm sistemi domine eder.
- P99'u düşürmek için timeout sınırları, circuit breaker ve spekülatif yeniden denemeler kullanılır.

- Mean latency masks catastrophic tail latency anomalies and user dissatisfaction.
- In fan-out microservice topologies, P99 tail latencies compound exponentially.
- Mitigate tail latencies using strict deadlines, speculative retries, and circuit breakers.

4

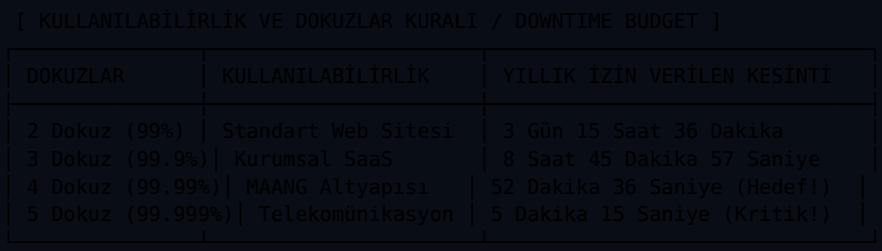
BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

Güvenilirlik ve Kullanılabilirlik (SLA, SLO, SLI & 9'lar Kuralı)

Reliability & Availability (SLA, SLO, SLI & The Rule of 9s)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir su tesisatçısı düşünün: 'Haftada bir gün musluktan su akmazsa sorun etmeyin' derse (%85 kullanılabilirlik) o tesisatçıyla çalışmazsınız. Ancak bir hastanede 'Yılda sadece 5 dakika elektrikler kesilebilir' derlerse (%99.999 - 5 dokuz), bu ameliyathanenin jeneratörlerle, çift hatlarla ve kusursuz bir mimariyle korunduğunu gösterir. MAANG mülakatında 'Sistemim %99.99 ayakta kalacak' dediğinizde, yılda sadece 52 dakika hata lüksünüz olduğunu bilerek konuşmalısınız!

🇬🇧 ENGLISH (ELI5)

Imagine a water utility saying: 'If water vanishes for an entire day every month, don't worry about it' (85% availability). You would fire them instantly! But an intensive-care hospital says: 'Power failure cannot exceed 5 minutes over an entire calendar year' (99.999% - Five Nines). That demands redundant generators, dual power grids, and fault-tolerant architecture. When promising 99.99% availability in a MAANG interview, you are budgeting a razor-thin 52 minutes of total annual downtime!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üç temel güvenilirlik kavramı: 1) SLI (Service Level Indicator): Gerçek zamanlı ölçülen metrik (ör. Başarılı HTTP isteklerinin yüzdesi). 2) SLO (Service Level Objective): Mühendislik ekibinin kendine koyduğu iç hedef (ör. Ayda %99.95 başarı). 3) SLA (Service Level Agreement): Müşteriye verilen yasal ve finansal taahhüt (ör. %99.9'un altına düşerse para iadesi). Hata Bütçesi (Error Budget): SLO'nuz %99.9 ise, %0.1'lik hata bütçeniz vardır. Bu bütçe tükenirse yeni özellik dağıtımları durdurulur ve sadece stabilite çalışılır.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The trifecta of Site Reliability Engineering: 1) SLI (Service Level Indicator): The empirical, real-time metric (e.g., successful request ratio over 30 days). 2) SLO (Service Level Objective): The internal target committed by the engineering team (e.g., 99.95% availability). 3) SLA (Service Level Agreement): The legally binding contractual commitment to paying clients with penalty credits. Error Budget: If your SLO is 99.9%, you have an error budget of 0.1%. When the error budget burns, feature deployments freeze in favor of reliability engineering.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: SLA & Hata Bütçesi (Error Budget) Hesaplama Motoru
public record AvailabilityCalculator(double availabilityPercentage) {
    private static final long SECONDS_IN_YEAR = 365L * 24 * 3600;

    public double calculateAnnualDowntimeSeconds() {
        double failureRate = (100.0 - availabilityPercentage) / 100.0;
        return SECONDS_IN_YEAR * failureRate;
    }

    public String formatDowntime() {
        long totalSecs = (long) calculateAnnualDowntimeSeconds();
        long hours = totalSecs / 3600;
        long mins = (totalSecs % 3600) / 60;
        long secs = totalSecs % 60;
        return hours + " sa " + mins + " dk " + secs + " sn";
    }
}

void main() {
    var fourNines = new AvailabilityCalculator(99.99);
    IO.println("99.99% Yıllık Maksimum Kesinti: " + fourNines.formatDowntime());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçınıza 'Sistemim %100 ayakta kalacak' demeyin! Dağıtık sistemlerde %100 kullanılabilirlik fiziksel olarak imkansızdır (ağ kabloları kopar, veri merkezleri su basar). %99.99 (Four Nines) gerçekçi kurumsal altın standarttır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never claim 100% availability in a system design interview. 100% is mathematically impossible across distributed networks. Stating 99.99% demonstrates architectural maturity.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

- SLI neyi ölçtüğünüz, SLO hedefiniz, SLA ise müşteriye verdiğiniz yasal sözdür.
- 99.99% (Dört Dokuz) yılda 52 dakikadan az kesinti demektir.
- Hata bütçesi (Error Budget), inovasyon hızı ile sistem stabilitesi arasındaki dengeyi sağlar.

- SLI is the metric, SLO is the internal goal, SLA is the contractual penalty threshold.
- Four Nines (99.99%) permits less than 52.6 minutes of downtime per year.
- Error budgets balance release velocity against platform stability.

5

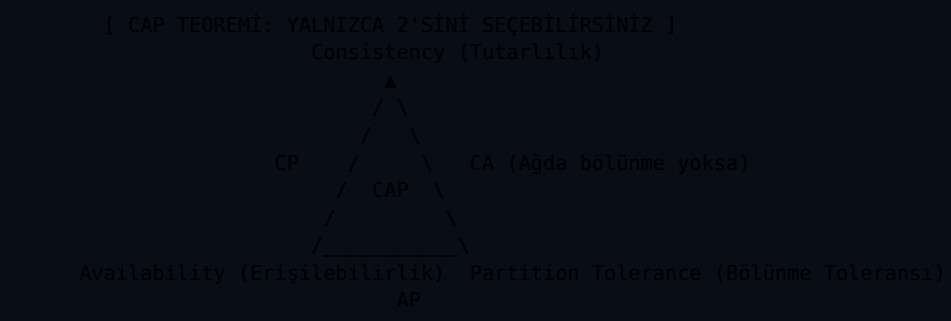
BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

CAP Teoremi ve PACELC: Ağ Bölünmesinde Tutarlılık mı, Erişilebilirlik mi?

CAP Theorem & PACELC: Consistency vs Availability

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bankanın Ankara ve İstanbul şubelerini düşünün. İki şube arasındaki telefon ve internet kablosu bir kaza sonucu koptu (Ağ Bölünmesi - Partition). Şimdi Ankara şubesine bir müşteri gelip hesabındaki son 1.000 TL'yi çekmek istiyor. Banka ne yapmalı? 1) 'İnternet koptu, işlem yapamam' deyip müşteriyi geri çevirirse Tutarlılığı (Consistency - CP) seçmiş olur. 2) 'Parayı verelim gitsin, kablo bağlanınca bakarız' derse Erişilebilirliği (Availability - AP) seçmiş olur ama müşteri aynı parayı İstanbul'dan da çekmiş olabilir! İkisini aynı anda garanti edemezsiniz!

🇬🇧 ENGLISH (ELI5)

Imagine two bank branches in New York and London. A submarine cable snaps, severing their communication line (Network Partition). A customer walks into London wanting to withdraw their last $1,000. What does the bank do? 1) Refuse the withdrawal until connection restores: chooses Consistency over Availability (CP). 2) Hand over the cash: chooses Availability over Consistency (AP), risking that the customer simultaneously withdrew the same $1,000 in New York! You cannot have both!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Eric Brewer'ın CAP Teoremi: Dağıtık bir veri deposu Consistency (Lineerize edilebilir anlık tutarlılık), Availability (Her sağlıklı düğümün hata yerine yanıt dönmesi) ve Partition Tolerance (Ağ paketleri kaybolsa da sistemin çalışması) özelliklerinden aynı anda ancak ikisini sağlayabilir. Dağıtık ağlarda kablo kopmaları ve gecikmeler kaçınılmaz olduğundan 'P' zorunludur; dolayısıyla gerçek dünya seçimi CP veya AP arasındadır. Daniel Abadi'nin PACELC teoremi bunu genişletir: Ağ bölünmesi yokken bile sistem Düşük Gecikme (Latency) mi yoksa Güçlü Tutarlılık (Consistency) mi hedefliyor? (Örn: MongoDB = PC/EC, Cassandra = PA/EL).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Brewer's CAP Theorem proves that a distributed data store can guarantee at most two of: Consistency (linearizability), Availability (every non-failing node returns a non-error response), and Partition Tolerance (network packet drops/splits). Since network partitions are an inevitable physical reality, 'P' is non-negotiable, boiling trade-offs down to CP vs AP. Abadi's PACELC expands this: If partitioned (P), trade Availability (A) vs Consistency (C); Else (E), trade Latency (L) vs Consistency (C). Example: Dynamo/Cassandra = PA/EL, HBase/Spanner = PC/EC.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: CAP/PACELC Modeli ve Quorum Tutarlılık Kontrolcüsü
public class QuorumConsistencyChecker {
    // R (Okuma Düğümü), W (Yazma Düğümü), N (Toplam Replika)
    // Güçlü Tutarlılık Kuralı: R + W > N
    public static boolean isStronglyConsistent(int totalNodes, int writeQuorum, int readQuorum) {
        return (writeQuorum + readQuorum) > totalNodes;
    }

    public static void main(String[] args) {
        int N = 3; // 3 Replikasyonlu Küme
        int W = 2; // Çoğunluk yazma (Quorum Write)
        int R = 2; // Çoğunluk okuma (Quorum Read)

        boolean strong = isStronglyConsistent(N, W, R);
        IO.println("R=" + R + ", W=" + W + ", N=" + N + " -> Güçlü Tutarlı mı? " + strong);
        // R + W = 4 > 3 -> En az 1 düğüm hem en son yazmayı hem okumayı görür!
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçılar 'Ben hem %100 tutarlı hem de her durumda anında cevap veren bir sistem istiyorum' diyerek sizi tuzağa düşürmeye çalışır. Tuzağa düşmeyin: Ağ bölünmesinde CAP gereği birinden ödün vermek zorunda olduğunuzu netçe ifade edin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Interviewers often bait candidates by asking for 100% consistency AND 100% availability during network partitions. Stand firm: explain that network splits make this mathematically impossible under CAP.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Gerçek dünyada ağ bölünmeleri (P) kaçınılmazdır; seçim CP veya AP arasındadır.

• Finansal ve bakiye sistemleri CP'yi, sosyal medya akışları ve beğeniler AP'yi tercih eder.

• PACELC teoremi, sistemin normal zamanlardaki Latency vs Consistency takasını açıklar.

🎯 Key Takeaways for the Interview

• Network partitions (P) are unavoidable; real architectures choose between CP and AP.

• Financial ledger ledgers favor CP; social media feeds and likes favor AP.

• PACELC clarifies the everyday trade-off between latency and consistency when no partition exists.

6

BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

Arka Plan Tahminleri (Back-of-the-Envelope Estimation): QPS, RAM ve Disk

Back-of-the-Envelope Capacity Estimations (QPS, RAM, Storage)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ MAANG KAPASİTE TAHMİNİ MATRİSİ (CHEAT SHEET) ]

+-----------------------------+-----------------------------------------+
| 1 Gün = 86,400 Saniye       | Kolay hesap için ~100,000 (10^5) saniye al! |
| 1 Milyon İstek / Gün        | ~12 QPS                                 |
| 100 Milyon İstek / Gün      | ~1,200 QPS (Zirve: ~2,400 QPS)          |
| 1 Milyar İstek / Gün        | ~12,000 QPS                             |
+-----------------------------+-----------------------------------------+
| VERİ BOYUTLARI              | 1 Char = 1 Byte | UUID = 16-36 B | Long = 8B |
| 100M tweet x 200 Byte       | 20 GB / gün -> 5 Yılda: ~36 TB depolama |
| %20 Önbellek Kuralı         | Günlük okuma verisinin %20'si RAM'de tutulur |
+-----------------------------+-----------------------------------------+
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir düğün organize ettiğinizi düşünün. Kaç gram et, kaç bardak su almanız gerektiğini hesaplamazsanız ya düğünün ortasında yemek biter rezil olursunuz ya da tonlarca et çürür paranız batar! Mülakatta da 'Twitter tasarlıyorum' dediğinizde mülakatçı kaç sunucuya, kaç terabayt diske ve ne kadar RAM'e ihtiyacınız olduğunu bilmek ister. Kimse sizden kuruşu kuruşuna kesin matematik beklemez; kabaca peçete arkasında yapılan (Back-of-the-envelope) mantıklı mühendislik tahminleri bekler!

🇬🇧 ENGLISH (ELI5)

Planning a wedding without estimating food means either guests starve midway through and riot, or you overbuy tons of meat that rots and bankrupts you! In a MAANG interview, if you say 'I will store all messages in memory', the interviewer will ask: 'How many gigabytes is that?' Nobody expects 5-decimal calculus; they expect rapid, disciplined back-of-the-envelope napkin math to sanity-check architectural feasibility!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında kapasite hesaplama matematiği: 1) Zaman Sabiti: 1 gün = 86,400 saniye ≈ 10^5 sn. 2) QPS Hesabı: 100M günlük istek / 100,000 sn ≈ 1,000 QPS (Zirve yük için 2x çarpanı alınır: 2,000 QPS). 3) Depolama Hesabı: Her kayıt 500 Byte ise, günlük veri = 100M x 500B = 50 GB/gün. 5 yıllık depolama = 50 GB x 365 x 5 ≈ 91 TB (3 kopya replikasyon ile 270 TB). 4) Önbellek RAM Hesabı (Pareto 80/20 Kuralı): İsteklerin %80'i verinin %20'sinden gelir. Günlük 50 GB verinin %20'si = 10 GB RAM önbellek yeterlidir.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Back-of-the-envelope capacity arithmetic: 1) Time approximation: 1 day = 86,400s ≈ 10^5s. 2) QPS: 100M requests/day ÷ 10^5s ≈ 1,000 QPS average (multiply by 2-3x for peak traffic = 2,500 QPS). 3) Storage: If an entity payload is 500 Bytes, 100M items/day = 50 GB/day. 5-year retention: 50 GB × 365 × 5 ≈ 91.25 TB (with 3x replication factor = 273 TB). 4) Memory Sizing (Pareto 80/20 Rule): 80% of read traffic targets 20% of data. Cache 20% of daily read volume: 50 GB × 0.20 = 10 GB Redis memory footprint.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Peçete Arkası Kapasite Tahmin Motoru (Capacity Estimator)
public record CapacityEstimator(long dailyRequests, int payloadBytes) {
    public long averageQps() {
        return dailyRequests / 86_400;
    }

    public long peakQps() {
        return averageQps() * 2;
    }

    public double dailyStorageGigabytes() {
        return (dailyRequests * (double) payloadBytes) / (1024 * 1024 * 1024);
    }

    public double fiveYearStorageTerabytes(int replicationFactor) {
        return (dailyStorageGigabytes() * 365 * 5 * replicationFactor) / 1024;
    }

    public double cacheMemoryGigabytes(double paretoRatio) {
        return dailyStorageGigabytes() * paretoRatio;
    }
}

void main() {
    var estimator = new CapacityEstimator(100_000_000L, 500);
    IO.println("Ortalama QPS:    " + estimator.averageQps());
    IO.println("Zirve (Peak) QPS: " + estimator.peakQps());
    IO.println("Günlük Depolama: " + String.format("%.2f GB", estimator.dailyStorageGigabytes()));
    IO.println("5 Yıllık (3x Rep):" + String.format("%.2f TB",
estimator.fiveYearStorageTerabytes(3)));
    IO.println("RAM Önbellek:    " + String.format("%.2f GB", estimator.cacheMemoryGigabytes(0.20)));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Hesaplama yaparken 86,400 ile tam bölmeye çalışıp vakit kaybetmeyin. Mülakatçıya 'Hesabı kolaylaştırmak için 1 günü 100.000 saniye kabul ediyorum' deyin; bu pratik mühendislik refleksiniz için artı puan kazandırır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not waste precious interview minutes dividing by 86,400 by hand. Explicitly tell the interviewer: 'I will approximate 86,400 as 100,000 seconds for rough sizing'; they will appreciate your pragmatism.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

- 1 gün ≈ 100,000 saniye kuralı QPS hesaplarını saniyeler içinde çözmenizi sağlar.
- Zirve (Peak) QPS her zaman ortalama QPS'in 2 ila 3 katı olarak kabul edilir.
- Pareto 80/20 kuralı sayesinde günlük verinin %20'sini önbelleğe almak yeterlidir.

- Use 1 day ≈ 10^5 seconds to compute baseline QPS within 5 seconds.
- Dimension service throughput for Peak QPS (typically 2x to 3x average QPS).
- The Pareto 80/20 heuristic sizes distributed cache memory at 20% of daily active working set.